Training Quantum Neural Networks with TensorFlow Quantum: a Small, Reproducible Study

Amal Mehta
Independent researcher

Abstract

Quantum neural networks (QNNs) are easy to describe and surprisingly hard to evaluate fairly. We present a small, fully reproducible study built on TensorFlow Quantum, aimed at readers who want to see the main behaviours of QNN training first-hand. A 17-qubit Farhi–Neven QNN learns to separate 3s from 6s in 4×4 binarised MNIST, reaching 0.868 test accuracy over 3 seeds. A classical network with 37 weights reaches 0.909, and adding an 8-unit classical head to the same circuit lifts it to 0.906. Both are close to the 0.916 that a lookup table over the training patterns sets as a ceiling for this input. Gradient variance in random circuits falls 27-fold from 2 to 8 qubits. Under depolarizing noise, models trained without noise drop to 0.770 at $p = 0.01$ and to chance at $p = 0.03$. Code, raw results and the scripts that generate every number in this paper are released.

1 Introduction

Parameterised quantum circuits trained by gradient descent are the most common model in near-term quantum machine learning [3, 11]. Software such as TensorFlow Quantum (TFQ) [2] lets anyone train them on a laptop. That makes it easy to run a demo, and also easy to misread one: without a matched classical baseline, a sense of what the data allows, or a look at gradients and noise, a working QNN says little.

This paper is a compact, reproducible tour of those checks on one task: the Farhi–Neven QNN [4] on 4×4 MNIST 3-vs-6, the setting of the TFQ tutorial. We contribute:

- a matched comparison of the QNN, a classical network of similar size, and a hybrid model that adds a classical head to the same circuit, alongside the accuracy ceiling the input allows (§4.1, §4.2);
- measurements of gradient variance in random circuits as width and depth grow (§4.3);
- the effect of depolarizing noise, both on trained models and during training (§4.4);
- code that regenerates every figure, table and number from raw results with one command, running on current TFQ (0.7.6) in Docker.

None of these phenomena are new. The point is to reproduce them together, with error bars, in a form that is easy to rerun and extend.

2 Background

Quantum neural networks. A parameterised quantum circuit (PQC) applies gates whose angles are trainable, and its output is the expectation value of an observable. Trained by gradient descent, PQCs are the near-term analogue of neural networks [3, 11]. Farhi and Neven [4] proposed a QNN for binary classification in which every data qubit couples to a single readout qubit, and the prediction is $\langle Z \rangle$ on that readout. We use their architecture throughout.

Gradients. On hardware, gradients come from the parameter-shift rule, which evaluates the circuit at shifted angles [10, 12]. In exact classical simulation, the adjoint method gives all gradients at roughly the cost of one forward and one backward pass [6].

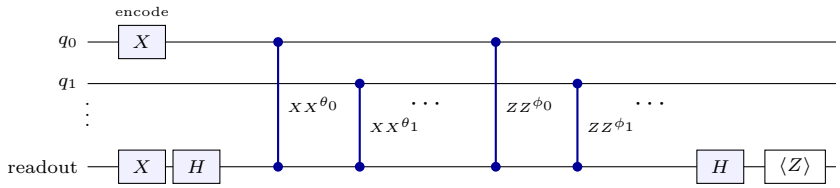


Figure 1: The QNN circuit, with two of the 16 data qubits drawn. Each data qubit couples to the readout qubit through one trainable XX gate and one trainable ZZ gate.

Barren plateaus. For sufficiently random circuits, the variance of the gradient with respect to any parameter decays exponentially in the number of qubits [9]. Hardware noise can induce the same effect even in shallow circuits [13].

TensorFlow Quantum. TFQ [2] represents circuits (built with Cirq) as tensors and exposes simulation, expectation values and their gradients as Keras layers on top of TensorFlow [1]. The same model code can use exact, noisy or sampled simulation. Its tutorials include a version of the Farhi–Neven MNIST task, which we follow for the data pipeline. Huang et al. [5] discuss when classical models can match quantum ones on data like this, which is relevant to our baseline.

3 Setup

Data. We keep the 3s and 6s from MNIST [8] and downsample each image to 4×4 by bilinear resizing. As in the TFQ tutorial, we then remove training images whose downsampled form occurs with both labels, and binarise every pixel at a threshold of 0.5. Each lit pixel becomes an X gate, so an image is the basis state $|x\rangle$ of 16 data qubits. Each seed draws 3000 training and 500 test images at random.

Models. *QNN.* A readout qubit is prepared with X then H. Every data qubit i then couples to it through XX^{θ_i}, then ZZ^{ϕ_i}, and a final H is applied to the readout (Fig. 1). The prediction is $\langle Z \rangle \in [-1, 1]$ on the readout, giving 32 parameters on 17 qubits. *Hybrid.* The same circuit, but $\langle Z \rangle$ on all 17 qubits feeds a dense layer with 8 ReLU units and a tanh output (185 parameters). *Classical.* A multilayer perceptron on the same 16 binary pixels, with 2 ReLU units and a tanh output (37 parameters), chosen to roughly match the QNN's parameter count.

Training. All models minimise the hinge loss on labels ± 1 with Adam [7] (learning rate 0.01, batch size 32) for 4 epochs, over 3 seeds. Quantum models are simulated exactly and differentiated with the adjoint method. We report mean $\pm$ standard deviation over seeds. We chose the batch size and learning rate from two candidate settings on seed 0.

Barren plateaus. Following McClean et al. [9], each random circuit applies $R_y(\pi/4)$ to every qubit, then layers of a uniformly random choice of R_x, R_y or R_z per qubit followed by a CZ ladder. We estimate $\mathrm{Var}[\partial \langle Z_0 Z_1 \rangle / \partial \theta_0]$ over 200 random circuits and angles, sweeping 2–12 qubits at 50 layers, and 1–50 layers at 8 qubits.

Noise. A single-qubit depolarizing channel with probability p follows every moment of the model circuit. Expectations average 50 quantum trajectories. We run two measurements. (i) The 4×4 QNNs trained above are tested under noise on 500 test images (a separate random draw from the one in Table 1, so the noise-free accuracies differ slightly). (ii) Models are trained and tested with the noise present. Training through trajectories needs parameter-shift gradients, which made 17 qubits too slow, so (ii) uses 3×3 images (10 qubits), 200 training and 200 test images, 3 epochs, batch size 16 and learning rate 0.02.

Table 1: Test accuracy on 4×4 binarised MNIST 3-vs-6 (mean $\pm$ std over 3 seeds, 3000 training and 500 test images per seed, 4 epochs). A lookup table over training patterns reaches 0.916. Training time is the mean wall-clock time of simulated training on CPU. It varied by up to 3× across seeds with machine load.

Model	Parameters	Test accuracy	Train time (s)
QNN (quantum readout)	32	0.868 ± 0.023	543
QNN + dense head (hybrid)	185	0.906 ± 0.006	636
classical MLP	37	$\mathbf{0.909 \pm 0.006}$	5

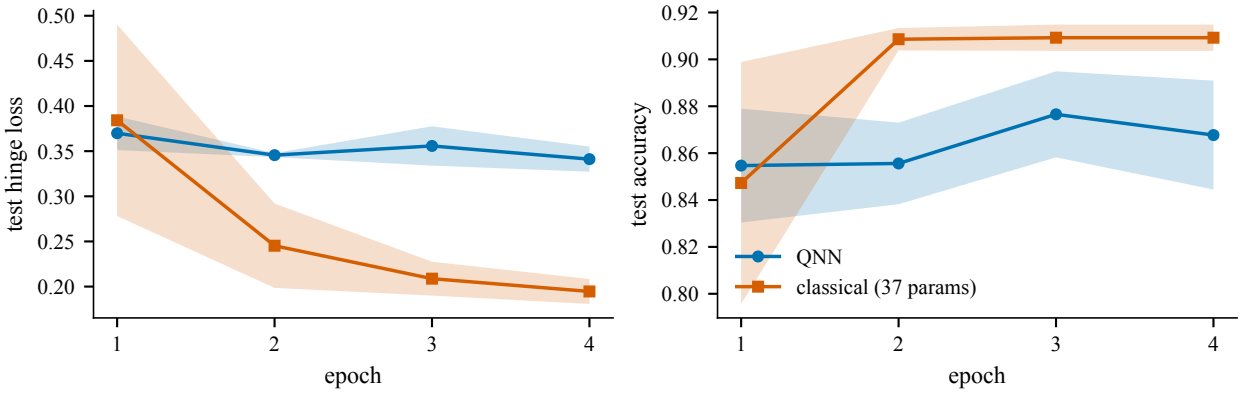


Figure 2: The classical network overtakes the QNN after the first epoch and ends higher, with less spread across seeds. Test hinge loss and accuracy per epoch, mean $\pm$ std over 3 seeds.

Software and hardware. TensorFlow 2.18.1, TensorFlow Quantum 0.7.6 and Cirq 1.5.0, in a Linux Docker container with 8 CPU cores on an Intel laptop, with no GPU.

4 Results

4.1 The QNN learns the task, and a classical network of similar size does slightly better

The QNN reaches 0.868 ± 0.023 test accuracy (Table 1, Fig. 2). The classical network, with 37 parameters against the QNN's 32, reaches 0.909 ± 0.006. Both are bounded by the input: binarising to 4×4 leaves 193 distinct training patterns, and predicting each pattern's majority label scores 0.916 on the full test set. The classical model sits almost on that ceiling, so on this input there is little room for any model to do better. The QNN also took about 543 s to train in simulation, against 5 s for the classical network.

4.2 A classical head on the readouts closes the gap

Measuring $\langle Z \rangle$ on all 17 qubits and passing the values through an 8-unit dense layer raises accuracy by 3.8 points, to 0.906 ± 0.006 (Fig. 3). This matches the classical model. The gain cannot be attributed to the circuit alone: the hybrid model has 185 parameters, most of them classical, and 16 of its 17 readouts are on data qubits, which start in the input's basis state.

4.3 Gradients shrink as random circuits grow

At 50 layers, the gradient variance falls from 8.6×10^{-2} at 2 qubits to 3.2×10^{-3} at 8 qubits, a factor of 27 (Fig. 4, left). Beyond 8 qubits it stops falling in our sweep, reaching 2.8×10^{-3} at 12 qubits, so a single exponential (decay factor 0.71 per qubit) describes the trend only loosely. The depth sweep suggests why. At 8 qubits the variance is still falling at 50 layers, 38 times below its single-layer value (Fig. 4, right), so

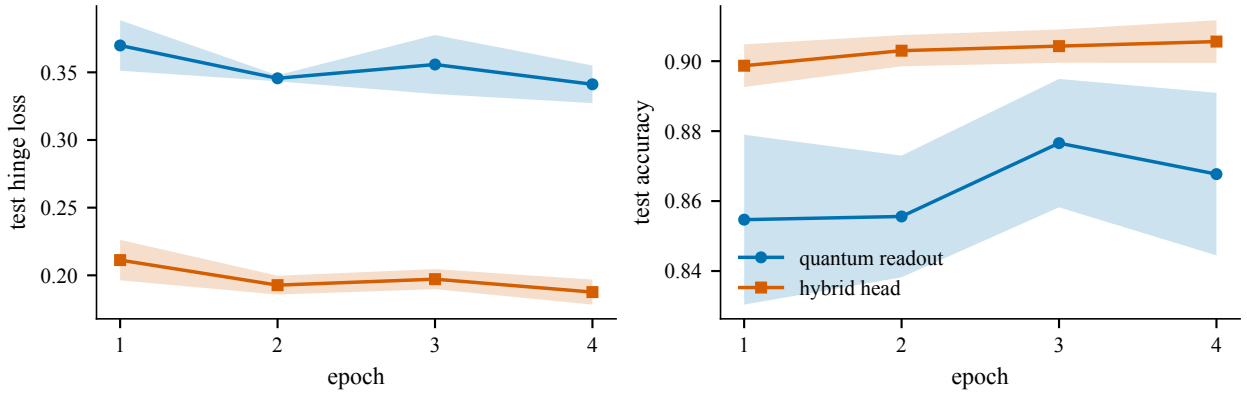


Figure 3: Reading out every qubit and adding a small dense head improves accuracy and reduces variance across seeds. Same data, seeds and circuit as Fig. 2.

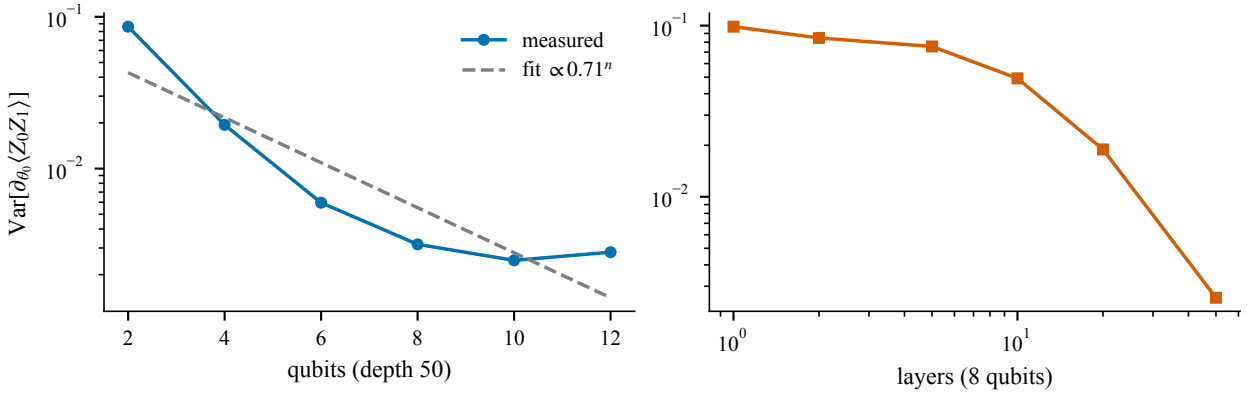


Figure 4: Gradient variance falls with width and with depth. Left: variance of $\partial\langle Z_0 Z_1\rangle/\partial\theta_0$ against qubit count at 50 layers, with a log-linear fit over all points. Right: against depth at 8 qubits. Each point uses 200 random circuits.

50 layers are probably not enough for the wider circuits to behave as random as the theory assumes [9]. We did not test this directly.

4.4 Depolarizing noise erodes accuracy quickly

The 4×4 QNNs from §4.1, tested under noise, go from 0.859 without noise to 0.770 at $p = 0.01$ and 0.501 at $p = 0.03$, which is chance (Fig. 5, Table 2). The channel acts after every moment of a circuit with 32 two-qubit couplings, so errors accumulate before the readout is measured. The 3×3 models trained with the noise present keep 0.859 at $p = 0.01$, against 0.862 without noise, and drop to 0.753 at $p = 0.03$. This comparison is confounded: the 3×3 circuit has 18 couplings rather than 32, so it accumulates less noise whether or not it is trained with it. Separating the effect of noise-aware training from circuit size would need noisy training at 4×4, which was too slow for us (§3).

5 Limitations

Everything here is classical simulation of small circuits. The task (two digits at 4×4 binary pixels) is far from where a quantum advantage could appear, and the input is so coarse that even a lookup table over training patterns sets a ceiling well below standard MNIST accuracies. We ran 3 seeds and lightly tuned two hyperparameters on one seed, so small differences between models should not be over-read. The noisy-

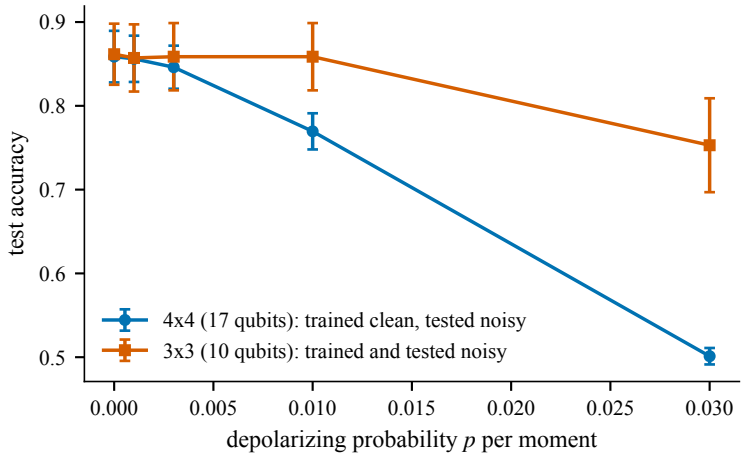


Figure 5: Trained 4×4 QNNs fall to chance by $p = 0.03$. Smaller 3×3 models trained with the noise present hold their accuracy up to $p = 0.01$. The two curves are different models with different circuit depths, so compare slopes, not heights. Mean $\pm$ std over 3 seeds; 50 trajectories per expectation.

Table 2: Test accuracy under depolarizing noise with probability p per qubit per moment (mean $\pm$ std over 3 seeds).

p	3×3, trained with noise	4×4, trained clean, tested with noise
0.0	0.862 ± 0.036	0.859 ± 0.031
0.001	0.857 ± 0.040	0.856 ± 0.028
0.003	0.859 ± 0.040	0.846 ± 0.026
0.01	0.859 ± 0.040	0.770 ± 0.022
0.03	0.753 ± 0.056	0.501 ± 0.010

training study uses smaller 3×3 images than the rest of the paper, and every noisy expectation carries trajectory-sampling error on top of the channel itself. The barren-plateau sweep stops at 12 qubits, with 200 circuits per point. None of these results are new. They reproduce known behaviour with current open-source tools.

6 Conclusion

TensorFlow Quantum makes the full loop (encoding classical data as circuits, training a QNN as a Keras layer, switching to noisy simulation, and probing gradient statistics) a few hundred lines of Python that run on a laptop CPU. On this small task, a classical model of similar size matches or beats the QNN, a classical head on the circuit's readouts brings the QNN up to that baseline, gradient variance shrinks as random circuits grow, and depolarizing noise quickly erodes a trained model's accuracy. These are the behaviours a newcomer to quantum machine learning should expect to see.

Reproducibility. All code, configurations and raw results ship with this paper. One command (`python -m quantumtf_demo.run_all --preset full`) reruns every experiment in Docker. Every number in the text is generated from the result files by a script, and `sources.md` records where each value comes from.

Use of AI tools

An AI assistant (Claude) wrote the experiment code, ran the experiments, generated the figure and table scripts, drafted the text and checked the references against arXiv and Crossref. The author chose the scope

and the experiments. **[TODO: state what the author reviewed and edited before release.]**

References

[1] Martín Abadi et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint*, 2016.

[2] Michael Broughton, Guillaume Verdon, Trevor McCourt, et al. Tensorflow quantum: A software framework for quantum machine learning. *arXiv preprint*, 2020.

[3] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021. doi: 10.1038/s42254-021-00348-9.

[4] Edward Farhi and Hartmut Neven. Classification with quantum neural networks on near term processors. *arXiv preprint*, 2018.

[5] Hsin-Yuan Huang, Michael Broughton, Masoud Mohseni, Ryan Babbush, Sergio Boixo, Hartmut Neven, and Jarrod R. McClean. Power of data in quantum machine learning. *Nature Communications*, 12: 2631, 2021. doi: 10.1038/s41467-021-22539-9.

[6] Tyson Jones and Julien Gacon. Efficient calculation of gradients in classical simulations of variational quantum algorithms. *arXiv preprint*, 2020.

[7] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint*, 2014.

[8] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

[9] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9:4812, 2018. doi: 10.1038/s41467-018-07090-4.

[10] Kosuke Mitarai, Makoto Negoro, Masahiro Kitagawa, and Keisuke Fujii. Quantum circuit learning. *Physical Review A*, 98:032309, 2018. doi: 10.1103/PhysRevA.98.032309.

[11] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018. doi: 10.22331/q-2018-08-06-79.

[12] Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99:032331, 2019. doi: 10.1103/PhysRevA.99.032331.

[13] Samson Wang, Enrico Fontana, M. Cerezo, Kunal Sharma, Akira Sone, Lukasz Cincio, and Patrick J. Coles. Noise-induced barren plateaus in variational quantum algorithms. *Nature Communications*, 12: 6961, 2021. doi: 10.1038/s41467-021-27045-6.